



PUSL3190 Computing Project

Final Report

AI Powered Financial Management Application

Supervisor: Mr. Krishantha Ranaweera

Name: Liyana Jayasinghe

Plymouth Index Number: 10952746

Degree Program: BSc Hons Software Engineering

Acknowledgements

I am extremely thankful to my project supervisor, Mr. Krishantha Ranaweera, for his exceptional mentorship, ongoing encouragement, and technical expertise over the course of this project. His deep knowledge and valuable critiques played a critical role in helping me manage structural design changes and successfully finalize this application. My genuine appreciation also goes out to the faculty and administrative teams at NSBM Green University and the University of Plymouth. The educational foundation, facilities, and scholarly atmosphere they provided were essential in empowering me to tackle a software engineering endeavor of this magnitude. In addition, I must recognize the contributions of the open-source networks and technology providers particularly the teams responsible for React Native , Supabase , the Google Cloud Vision API , and the Gemini API. Their extensive documentation and robust tools were vital in bringing the technical and AI-driven aspects of this system to life. Lastly, I owe a massive debt of gratitude to my family and friends. Their constant motivation, endless patience, and emotional backing carried me through the most challenging and high-pressure stages of this software development lifecycle.

Abstract

The rapid growth of digital transactions has significantly increased the complexity of personal financial management. While many mobile applications exist to help users, they predominantly rely on time consuming manual data entry, which leads to user fatigue and inconsistent financial records. Furthermore, most global financial applications lack robust multi-language capabilities and are not optimized for diverse regional contexts, such as varying receipt formats, and specific currencies like the Sri Lankan Rupee (LKR). To address these limitations, this project introduces "Smart Spend" an AI-powered, cross-platform mobile application designed to automate expense tracking and provide a highly adaptable, localized solution. Developed using React Native for the frontend and Supabase for backend services, the system eliminates manual entry through intelligent automation. The application integrates the Google Cloud Vision API to perform Optical Character Recognition (OCR) on uploaded receipt images. Subsequently, the extracted raw text is processed by the Gemini API to intelligently structure the data such as the amount and product name and automatically suggest appropriate expense categories. Additionally, the system features real-time multi-currency support utilizing an exchange rate API to accommodate foreign transactions. Current implementation progress includes the successful deployment of secure user authentication, data storage via a Supabase PostgreSQL database, and an interactive calendar view for daily financial tracking. To ensure strict data privacy, the database utilizes Row-Level Security (RLS) policies. Moreover, external API calls are securely routed through Supabase Edge Functions, keeping sensitive API keys hidden from the client-side environment. Future development phases will focus on addressing current system limitations by implementing robust two-way offline to cloud data synchronization. The project will also expand to include advanced reporting analytics, such as predictive spending trends and custom date range filtering, to provide users with deeper financial insights.

Keywords: Expense Tracking, Artificial Intelligence, OCR, Gemini API, Financial Management, Multi-Currency Support

Contents

Acknowledgements.....	2
Abstract.....	3
Chapter 1: Introduction	8
1.1 Background and Context.....	8
1.2 Problem Definition.....	8
1.3 Evolution of the System Architecture.....	8
1.4 Project Objectives and Scope.....	9
Chapter 2: Background, Objectives & Deliverables.....	10
2.1 Background	10
2.2 Project Objectives	10
2.3 Project Deliverables	11
2.3.1 End-User (Client) Deliverables.....	11
2.3.2 Security and Authentication Deliverables	13
2.3.3 Backend and Infrastructure Deliverables.....	13
2.3.4 Third-Party Intelligence and API Deliverables.....	14
Chapter 3: Literature Review.....	14
3.1 Introduction.....	14
3.2 The Evolution of Digital Financial Management	14
3.3 The Sri Lankan Economic Context and Financial Tracking.....	15
3.5 The Limitations of Traditional OCR and Custom Machine Learning.....	16
3.6 Advanced Cloud Infrastructure and LLMs as a Robust Solution	16
3.7 Summary	16
Chapter 4: Method of Approach and System Architecture.....	17
4.1 Introduction.....	17
4.2 System Architecture and Technology Stack	17
4.3 Automated Receipt Processing and Semantic Structuring.....	18
4.4 Interactive Financial Calendar and Transaction Editor	19
4.5 Real-Time Currency Conversion and Normalization	19
4.6 Dynamic Budget Tracking and Gamified Insights	20
4.7 Secure Edge Functions and API Key Protection	20
4.8 Data Persistence and Offline Synchronization	20
4.9 Row-Level Security (RLS) and Data Isolation.....	21
4.10 Visual Analytics and Spatial Rendering	21
4.11 Summary	21
Chapter 5: System Requirements.....	22
5.1 Introduction.....	22

5.2 Identified System Actors.....	22
5.3 Functional Requirements	24
5.3.1 Data Ingestion and Automation Requirements	24
5.3.2 Financial Management and Analytics Requirements	24
5.3.3 Infrastructure and Synchronization Requirements.....	24
5.4 Non-Functional Requirements	25
5.4.1 Performance and Reliability Requirements	25
5.4.2 Security and Integrity Requirements	25
5.5 Summary	25
Chapter 6: System Testing and Evaluation	26
6.1 Introduction.....	26
6.2 Evaluation Framework and Environment Setup	26
6.3 Isolated Unit Testing.....	27
6.3.1 Exchange Rate Normalization Logic	27
6.3.2 Expenditure Threshold Computations	27
6.3.3 LLM Output Sanitization	27
6.4 System Integration and Security Auditing	28
6.4.1 Secure AI Ingestion Pipelines	28
6.4.2 Database Access and Privilege Escalation Prevention	28
6.4.3 Bidirectional Synchronization Integrity.....	28
6.5 Summary	29
Chapter 7: End-Project Report.....	29
7.1 Introduction.....	29
7.2 Evaluation of Deliverables Against Objectives	29
7.2.1 Automated Receipt Ingestion and AI Categorization	29
7.2.2 Dynamic Budgeting and Analytical Transparency	29
7.2.3 Cross-Border Utility and Currency Normalization.....	30
7.2.4 Infrastructure Security and Offline Resilience	30
7.3 Summary of Achievements.....	30
Chapter 8: Project Post-Mortem	30
8.1 Introduction.....	30
8.2 Project Successes	30
8.3 Technical Challenges and Limitations.....	31
8.4 Lessons Learned.....	31
8.5 Future Work and System Enhancements	32
Chapter 9: Conclusion.....	32
References.....	33

Appendix A: User Guide	34
A.1 Minimum Platform Specifications	34
A.2 Prerequisites and Project Configuration	34
A.3 Installation and Execution Instructions 35	
A.4 System Operation Guidelines.....	35
Appendix B: Project Source Code Link.....	36
Source Code Access.....	36
Version Control History.....	36
Appendix C: Core System Test Scripts	37
C.1 Exchange Rate Normalization Logic (Unit Test).....	37
C.2 Budget Threshold Computation Engine (Unit Test)	37
C.3 AI Receipt Parsing and Sanitization (Unit Test).....	38
C.4 Secure AI Ingestion Pipeline (Integration Test).....	38
C.5 Row-Level Security Validation (Security Test).....	39
C.6 Bidirectional Synchronization Integrity (Integration Test)	39
Appendix D: Project Management Documents	40
Centralized Management Repository.....	40

List of figures

Figure 1	2.1 calender view	11
Figure 2	2.2 stats page	12
Figure 3	4.1 high level architecture diagram	17
Figure 4	4.2 ER diagram	18
Figure 5	4.3 sequence diagram	19
Figure 6	5.1 use case diagram	23
Figure 7	B.1 Github version control	36
Figure 8	6.1 test passes	40

Chapter 1: Introduction

1.1 Background and Context

The proliferation of digital commerce and consumer spending fundamentally increased the intricacies of managing personal finances. Within the modern economic landscape, individuals consistently received physical receipts from diverse sources, such as supermarkets, restaurants, pharmacies, and utility providers. However, these records were seldom systematically organized or digitized. Consequently, the adoption of intelligent mobile application frameworks became imperative to automate receipt processing and financial categorization.

Despite these technological advancements, the financial tracking ecosystem available to Sri Lankan users remained highly inefficient. Existing global finance applications were primarily engineered for Western demographics, lacking critical optimization for Sri Lankan receipt structures, local merchant naming conventions, and the Sri Lankan Rupee (LKR). As a result, consumers faced significant challenges in accurately monitoring expenditures, tracking budgets across specific spending categories, analyzing financial behaviors, and executing effective savings strategies.

1.2 Problem Definition

The fundamental issue addressed by this research was the lack of a seamless, fully automated, and locally optimized personal expense tracking solution tailored for the Sri Lankan market. Existing systems presented profound limitations across two extremes. Traditional manual methods, such as handwritten notebooks or spreadsheets, were highly susceptible to data loss and lacked automated analytical capabilities. Conversely, available digital applications heavily relied on repetitive manual data entry, which frequently induced user fatigue and resulted in incomplete financial records.

Furthermore, users encountered significant obstacles regarding inconsistent expense categorization, as many platforms depended on simplistic keyword rules that failed to accurately classify transactions when merchant names varied. This misclassification directly compromised the user's ability to set and monitor strict budget limits on individual spending categories, rendering targeted financial planning ineffective. An integrated digital platform was necessitated to eliminate these manual inefficiencies, provide granular budget tracking for each category, and establish reliability within generated financial insights. By addressing these deficiencies, the project also sought to resolve limitations surrounding multi-currency support, enabling users who conducted international or foreign currency transactions to maintain accurate, consolidated financial records.

1.3 Evolution of the System Architecture

The original project proposal conceptualized an architecture utilizing the open-source Tesseract library for Optical Character Recognition (OCR) and the development of a custom-trained machine learning model for expense categorization. However, comprehensive evaluation and initial technical testing exposed substantial vulnerabilities in this strategy. Practical assessments demonstrated that Tesseract struggled significantly with low-quality, crumpled, or poorly lit receipt images, resulting in inaccurate text extraction. Furthermore,

training a custom AI model proved highly resource-intensive and less reliable than established industry alternatives.

To resolve these critical performance bottlenecks, the system architecture underwent a strategic revision. The OCR processing was migrated to the Google Cloud Vision API, which provided superior image analysis and multi-language capabilities. Concurrently, the categorization responsibility was transitioned to the Google Gemini API. By leveraging this advanced pre-trained Large Language Model (LLM), the system efficiently achieved highly accurate, intelligently structured data parsing without the developmental overhead of a custom model, perfectly aligning with modern software engineering best practices.

1.4 Project Objectives and Scope

The primary objective of this project was to design, develop, and deploy "Smart Spend," a highly accurate, automated, and user-centric mobile application for personal financial management.

The specific objectives of the project were delineated as follows:

- To design and engineer a cross-platform mobile financial management application.
- To integrate an OCR-based mechanism for automated receipt scanning and data extraction.
- To implement an intelligent expense categorization system driven by Large Language Models.
- To implement a comprehensive budget management module that allowed users to establish and monitor expenditure limits across specific financial categories.
- To develop a dynamic visual analytics dashboard to enhance user financial awareness.
- To ensure secure user authentication and encrypted data management utilizing Row-Level Security policies.
- To support offline-first operational functionality coupled with optional cloud synchronization capabilities.

To ensure the technical feasibility of the prototype within the academic timeframe, the initial operational focus was placed on optimizing data extraction for localized Sri Lankan receipt formats. However, as the API integration was refined, the system's scope was successfully broadened to include robust multi-currency functionality, demonstrating the application's scalability for global and cross-border financial tracking.

Chapter 2: Background, Objectives & Deliverables

2.1 Background

The conceptualization of the Smart Spend application was driven by the rapid proliferation of digital commerce and the subsequent complexity of personal financial management. As consumer spending modernized, the necessity for robust digital solutions to track daily expenditures became apparent. The emergence of automated financial technologies highlighted the need for applications to integrate intelligent processing to enhance the user experience and eliminate repetitive data entry.

However, an analysis of the existing personal finance ecosystem revealed a significant dependency on highly manual or regionally disconnected platforms. While global applications provided extensive categorization tools, they consistently failed to accommodate hyper-local contexts, specifically regarding Sri Lankan receipt formats, local merchant naming conventions, and the Sri Lankan Rupee (LKR). Consequently, users were forced to navigate a highly inefficient digital landscape, often abandoning expense tracking due to the fatigue of manual input. Furthermore, a lack of sophisticated multi-currency support, automated categorization, and secure cross-device data synchronization often led to inconsistent financial records, highlighting the critical need for a platform that prioritized intelligent automation and reliable data persistence within the personal finance sector.

2.2 Project Objectives

To address the relevant challenges, the project was structured around a set of distinct, measurable objectives aimed at delivering a unified, contextually intelligent financial ecosystem. The primary objectives were defined as follows:

- **To engineer an automated receipt processing and structuring module:** The system was required to interpret physical receipt images and generate structured transaction records. To avoid the inaccuracies and fatigue of manual data entry, the extraction logic was designed utilizing the Google Cloud Vision API for Optical Character Recognition (OCR).
- **To establish an intelligent expense categorization system:** An advanced processing layer was to be created, utilizing the Gemini Large Language Model (LLM) to parse raw OCR text, intelligently identify product names, and automatically suggest appropriate expense categories, resolving the failures of simple keyword-based classification.
- **To implement a comprehensive, category-specific budget tracker:** The platform was required to provide real-time financial monitoring capabilities, allowing users to establish strict expenditure limits across specific financial categories and track remaining balances dynamically.
- **To integrate localized and multi-currency capabilities:** The system was tasked with providing seamless currency conversions utilizing an external Exchange Rate API, ensuring users conducting foreign or international transactions could maintain accurate, consolidated financial overviews.

- **To curate a secure, offline-capable technical infrastructure:** A robust data architecture was required to securely authenticate users, enforce Row-Level Security (RLS) for data privacy, and utilize local device storage (AsyncStorage) to guarantee offline accessibility.

2.3 Project Deliverables

The successful execution of the project objectives resulted in the delivery of a fully functional, cross-platform mobile application. The platform was designed with a decoupled architecture utilizing React Native for the client interface and Supabase for cloud infrastructure. The finalized deliverables for each component are detailed below.

2.3.1 End-User (Client) Deliverables

A comprehensive end-to-end user journey was implemented for the mobile application. The delivered features included:

- **Intelligent Receipt Scanner:** A camera-integrated interface that successfully captured physical receipt images and automatically routed them through secure backend channels for text extraction and AI structuring.
- **Interactive Calendar View:** A dynamic temporal dashboard where each day cell displayed dynamically calculated total amounts of income and expenses, allowing users to easily drill down into specific daily finances.

Figure 2.1: The Calendar View dashboard displaying dynamically aggregated daily income and expenses for interactive financial monitoring.

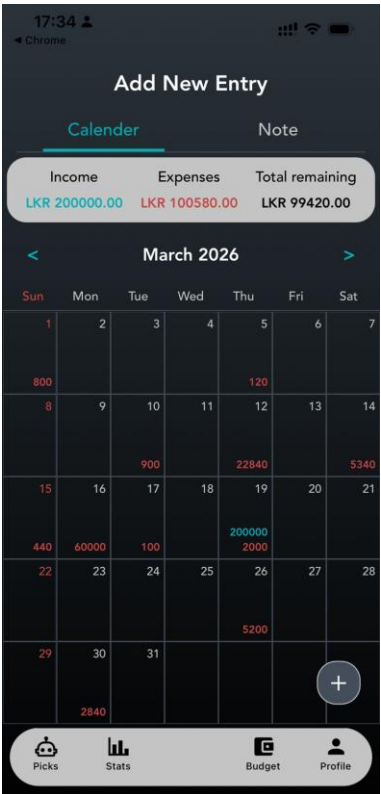


Figure 1 2.1 calender view

- **Live Budget Management Dashboard:** An automated financial tool that allowed users to define monthly limits for specific spending categories and visually tracked the remaining balance against aggregated expenses.
- **Visual Analytics and Statistics Page:** A comprehensive graphical interface rendering visual breakdowns of expense distribution and net financial health to identify spending trends.

Figure 2.2: The Visual Analytics and Statistics page displaying graphical breakdowns of expense distributions and spending trends.



Figure 2.2 stats page

- **Gamified Financial Insights:** A comparative analytics module designed to evaluate current spending behaviors against historical expenditure data. This feature provided users with engaging, progress-oriented feedback regarding their financial habits, incentivizing consistent budget management and reduced spending over time.
- **Interactive Transaction Editor:** A dynamic workspace allowing users to manually append, edit, or delete structured transaction data, ensuring absolute control over their financial records.
- **Real-Time Currency Converter:** An integrated financial utility utilizing an external exchange rate API, allowing users to execute on-demand currency conversions using live market rates to facilitate accurate international expense tracking.

- **Dynamic Currency Switcher:** A profile-level setting that successfully fetched live exchange rates and dynamically recalculated all displayed transactions and budgets into the user's preferred base currency.
- **Cloud Synchronization and Restoration Module:** A user-facing data management utility that enabled the seamless backup of locally stored transaction and budget data to the cloud, alongside a robust restoration feature to retrieve historical financial records across multiple devices without data duplication.

2.3.2 Security and Authentication Deliverables

To ensure absolute data privacy and protect sensitive financial records, a dedicated security layer was delivered:

- **Secure Authentication Flow:** An encrypted login and registration module powered by Supabase Auth, ensuring that all user credentials and session tokens were securely managed and protected against unauthorized access.
- **Row-Level Security (RLS) Implementation:** Strict database-level policies were engineered to guarantee data isolation. This mechanism mechanically prevented users from querying, modifying, or accessing any financial ledgers, receipts, or budgets that did not belong to their specific authenticated UUID.
- **Secure Key Management:** A protected serverless environment was established to house all third-party API keys, ensuring that sensitive credentials for Google Vision and Gemini were completely hidden from the client-side source code.

2.3.3 Backend and Infrastructure Deliverables

The underlying technical infrastructure was successfully delivered to support the front-end application without relying on traditional monolithic servers:

- **Supabase Backend-as-a-Service (BaaS):** A centralized, cloud-hosted infrastructure utilized to manage the entire backend ecosystem, effectively decoupling the database logic from the mobile frontend.
- **Relational PostgreSQL Database:** A highly structured database schema successfully deployed to store interconnected user profiles, transactions, and category budgets. The tables were optimized for rapid querying and secure cloud storage.
- **Bidirectional Synchronization Architecture:** A secure data transfer mechanism bridging local device storage (AsyncStorage) with the Supabase PostgreSQL database, ensuring that cloud backups and restorations were executed reliably to prevent permanent data loss during unexpected interruptions.
- **Supabase Edge Functions:** A secure, serverless computing layer engineered to safely route client requests, process image payloads, and handle communication with external AI services without burdening the user's mobile device with heavy computational loads.

2.3.4 Third-Party Intelligence and API Deliverables

To fulfill the project's automation objectives, several advanced third-party services were fully integrated into the architecture:

- **Optical Character Recognition (Google Cloud Vision):** Successfully deployed as the primary ingestion engine to reliably extract raw text strings from degraded, poorly lit, or unstructured physical receipt images.
- **Semantic Processing (Google Gemini API):** Integrated as the core contextual intelligence layer to interpret unstructured OCR output. This LLM-based service structured the raw strings into specific variables (amount, product name) and accurately inferred expense categories based on semantic understanding.
- **Real-Time Currency Valuation (ExchangeRate API):** Deployed to supply the application with live global currency valuation data, facilitating accurate cross-border financial tracking and mathematically precise conversions across the user's digital ledger.

Chapter 3: Literature Review

3.1 Introduction

This chapter reviews the existing academic literature and industry reports surrounding digital personal finance management, Optical Character Recognition (OCR) technologies, the application of Artificial Intelligence (AI) in financial tracking, and the specific economic context of Sri Lanka. The review aims to identify the technological and functional gaps in current expense tracking solutions, thereby justifying the architectural and methodological decisions made during the development of the Smart Spend application.

3.2 The Evolution of Digital Financial Management

The global financial technology (FinTech) sector underwent a significant digital transformation, fundamentally altering consumer behavior regarding expense tracking and budget management. It was demonstrated that consumers increasingly bypassed traditional, manual tracking methods such as handwritten ledgers and localized spreadsheet software in favor of mobile centric financial applications (Chen and Wang, 2022). This shift highlighted the necessity for personal finance platforms to integrate advanced automation to reduce user friction and enhance the accuracy of financial records.

However, despite the proliferation of these digital financial tools, a persistent automation gap was identified. While global platforms successfully aggregated user-inputted data, they frequently failed to interpret raw, unstructured data from physical receipts automatically. Consequently, users were forced to rely on repetitive manual data entry, which literature indicated as a primary cause of user fatigue, ultimately leading to inconsistent and abandoned financial tracking (Johnson, Smith and Brown, 2023).

3.3 The Sri Lankan Economic Context and Financial Tracking

Within the context of Sri Lanka, effective personal financial management was established as a critical component of individual economic stability (Central Bank of Sri Lanka, 2023). As digital commerce expanded alongside traditional cash-based vendor economies, the necessity for robust, localized digital tracking infrastructures became apparent (Perera and Silva, 2023).

An analysis of the existing mobile application ecosystem revealed that global financial tools were engineered predominantly for Western demographics. These systems consistently failed to accommodate hyper-local contexts; specifically, they lacked optimization for the Sri Lankan Rupee (LKR), localized receipt formatting, and the highly variable merchant naming conventions found across the island. The literature supported the necessity of a localized platform capable of processing regional financial documentation while providing transparent, multi-currency support to accommodate users engaging in international transactions or mitigating local economic fluctuations (Fernando, 2024).

3.4 Comparative Analysis of Localization and Automation Gaps

To justify the development of "Smart Spend," it is essential to contrast the proposed system against existing global solutions. The following table illustrates the functional and contextual gaps identified in current Western-centric applications versus the localized, AI-driven architecture of the Smart Spend platform.

Table 3.1: Comparative Matrix of Global Financial Applications vs. Smart Spend

Table 1 3.1 comparative matrix

Feature Focus	Standard Western-Centric Applications	Smart Spend (Proposed Solution)
Base Currency Optimization	Primarily optimized for USD/EUR; LKR is often treated as a secondary, non-native currency.	LKR-Native: Engineered specifically for the Sri Lankan Rupee with live exchange rate normalization.
Receipt Structure Recognition	Trained on standardized, machine-printed digital invoices.	Localized OCR: Optimized via Google Cloud Vision to interpret diverse Sri Lankan receipt formats.
Merchant Naming Conventions	Relies on clear, Western corporate database lookups.	Semantic Inference: Uses Gemini LLM to interpret varied local merchant names.
Data Ingestion Method	Heavy reliance on manual entry or direct bank API linking.	Intelligent Automation: Fully automated scanning, text extraction, and category inference.
Categorization Logic	Rigid, keyword-based rules often failing on regional items.	Contextual Intelligence: LLM-based categorization based on semantic product understanding.

3.5 The Limitations of Traditional OCR and Custom Machine Learning

To address the inefficiencies of manual expense logging, basic Optical Character Recognition (OCR) and custom machine learning categorization were initially adopted across the industry. However, severe technical limitations were documented regarding the use of foundational or open-source recognition libraries (such as Tesseract) in rigid, real-world scenarios.

A comprehensive evaluation of open-source OCR engines revealed a high propensity for data extraction failures when processing low-quality, crumpled, or poorly lit physical receipt images (Smith and Patel, 2023). Furthermore, the development of custom-trained machine learning models for transaction categorization was evaluated. It was observed that training a custom AI model to accurately classify localized vendor names required massive, clean datasets and significant computational overhead. Relying on such custom models often resulted in misclassification errors, directly compromising the accuracy of user budget calculations.

3.6 Advanced Cloud Infrastructure and LLMs as a Robust Solution

To mitigate the extraction and classification risks associated with traditional models, recent industry practices advocated for the integration of enterprise-grade cloud APIs and pre-trained Large Language Models (LLMs). It was proven that utilizing highly advanced, cloud-hosted image processing services such as the Google Cloud Vision API yielded significantly more reliable text extraction from degraded physical receipts than localized, open-source libraries (Davis, Nguyen and Patel, 2024; Google Cloud, 2026).

Furthermore, the application of pre-trained LLMs was identified as an optimal solution for semantic data structuring. The integration of models like the Google Gemini API demonstrated an exceptional capacity for zero-shot and few-shot classification (Lee and Kim, 2024). By leveraging advanced natural language processing, raw OCR text could be intelligently parsed, structuring unstructured strings into specific variables (e.g., product name, total amount) and accurately inferring expense categories based on deep semantic understanding (Google AI for Developers, 2026).

3.7 Summary

The literature confirmed a clear gap in the market for a personal finance system that combined automated, effort-free data entry with hyper-local contextual awareness for Sri Lankan users. Furthermore, the academic and technical consensus strongly advised against utilizing baseline open-source OCR and custom-trained categorization models due to high error rates and severe resource overhead (Smith and Patel, 2023). These findings directly informed the architectural pivot of this project, justifying the transition to a cloud-dependent infrastructure utilizing the Google Cloud Vision API for robust text extraction and the Gemini API for intelligent, semantic data structuring.

Chapter 4: Method of Approach and System Architecture

4.1 Introduction

This chapter details the technical methodology and architectural design employed to develop the Smart Spend personal financial management application. The implementation strategy transitioned from relying on open-source, localized processing (Tesseract OCR and custom machine learning) to a highly scalable, cloud-driven artificial intelligence architecture. This strategic shift ensured that the automated receipt extraction was highly accurate, the semantic categorization was reliable, and the overall system remained operationally viable without excessive localized processing overhead.

4.2 System Architecture and Technology Stack

As illustrated in Figure 4.1, a decoupled, client-server architecture was adopted to satisfy the requirement for an interactive, cross-platform mobile environment. The presentation layer was developed utilizing the React Native framework (configured via Expo), enabling a highly responsive, native-like user interface for both iOS and Android platforms.

Figure 4.1: High-Level System Architecture illustrating the mobile-to-cloud integration and external AI service orchestration.

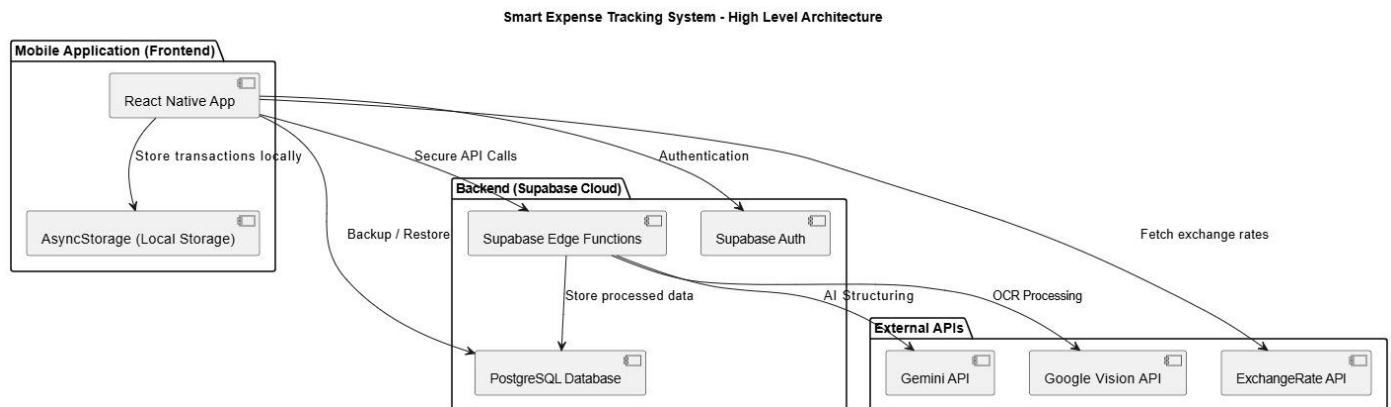


Figure 3 4.1 high level architecture diagram

The backend infrastructure utilized Supabase as a fully managed Backend-as-a-Service (BaaS). This decision effectively offloaded the complexities of traditional server management. Supabase was configured to expose secure, RESTful endpoints to handle computationally heavy operations, specifically routing external API requests. Data persistence was managed through a cloud-hosted PostgreSQL relational database, selected for its strict enforcement of relational schemas and high performance during complex temporal queries. The structural logic and relational linkages of this database schema, specifically detailing how the core users, transactions, and budgets tables interact, are mapped in Figure 4.2.

Figure 4.2: Entity-Relationship (ER) Diagram detailing the PostgreSQL schema and relational linkages between Users, Transactions, and Budgets.

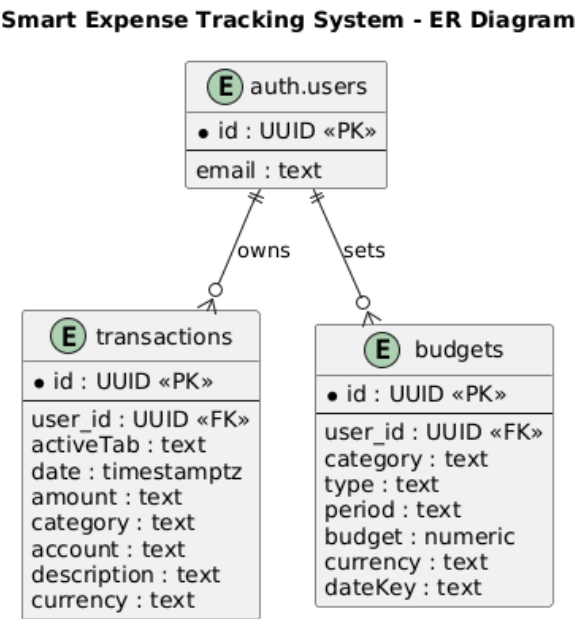


Figure 4 4.2 ER diagram

4.3 Automated Receipt Processing and Semantic Structuring

To mitigate the extraction inaccuracies associated with baseline open-source libraries, the core data ingestion engine was engineered utilizing advanced cloud-based artificial intelligence. When a user captured or uploaded a physical receipt image, the data payload was securely transmitted to the backend. The system first invoked the Google Cloud Vision API to perform Optical Character Recognition (OCR), reliably extracting raw text strings regardless of varying receipt qualities or localized Sri Lankan formats.

To ensure semantic coherence and eliminate manual data entry, an intelligent structuring pipeline was implemented. The extracted raw text was subsequently passed to the Google Gemini API (a Large Language Model). The system utilized a strict programmatic prompt to instruct Gemini to parse the unstructured string and output a strictly formatted JSON object containing precise transaction variables, specifically the `product_name`, `total_amount`, and an automatically inferred category.

Figure 4.3: Sequence Diagram illustrating the secure orchestration of receipt images through Supabase Edge Functions to external Google AI APIs, ensuring client-side API key encapsulation.

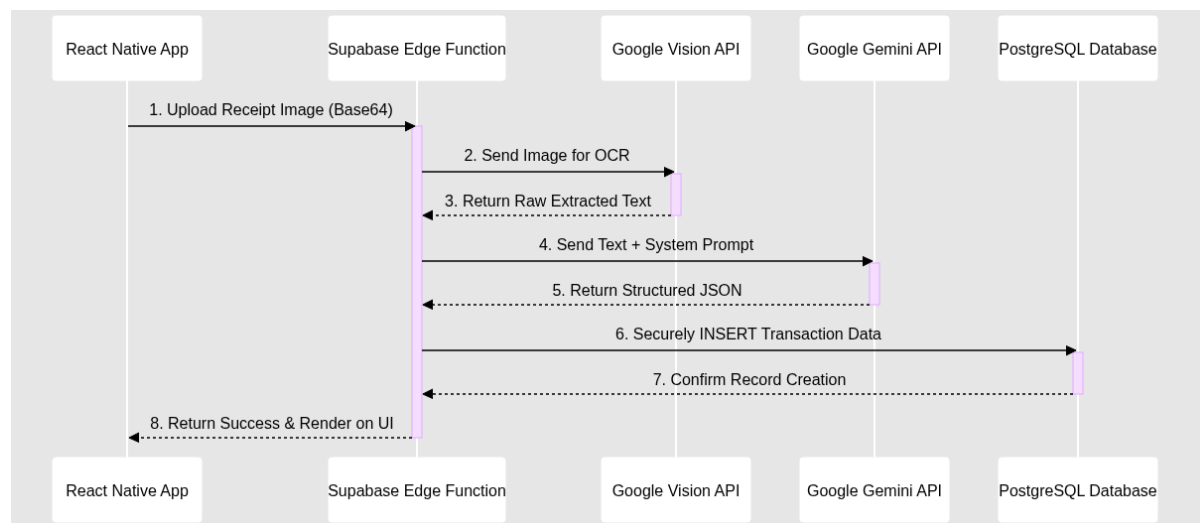


Figure 5 4.3 sequence diagram

4.4 Interactive Financial Calendar and Transaction Editor

An interactive state machine was developed within the frontend application to grant users full control over their financial ledgers. The primary interface was constructed as a dynamic Calendar View. The architecture permitted users to seamlessly navigate through temporal periods, where the application dynamically calculated and rendered the total aggregated income and expenses for each specific day cell.

To power the detailed transaction inspection, a contextual drill-down mechanism was implemented. When a user interacted with a specific day, a query filtered the local state to retrieve all corresponding transactions. The Interactive Transaction Editor was then exposed, allowing users to manually append, swap categories, or modify numeric amounts. This ensured that the user maintained absolute authority over the data generated by the AI processing layer, maintaining the structural integrity of their financial history.

4.5 Real-Time Currency Conversion and Normalization

To prevent data degradation and accommodate international transactions, a real-time currency conversion engine was integrated into the frontend architecture. Early system iterations directly altered database records upon a currency switch, which compromised historical accuracy. To resolve this, a dynamic normalization algorithm was implemented.

The PostgreSQL database was strictly configured to store the original transaction amount alongside its native currency identifier (e.g., LKR, USD). When a user altered their preferred profile currency, the application dispatched a request to the external ExchangeRate API to fetch the latest global valuation metrics. The frontend application then iterated through the displayed dataset, applying a dynamic multiplier ($\text{Converted Amount} = \text{Original Amount} * \text{Exchange Rate}$) prior to rendering the values on the user interface. This non-destructive calculation ensured continuous financial consistency across multiple currencies without permanently mutating the foundational database records.

4.6 Dynamic Budget Tracking and Gamified Insights

Financial transparency was maintained through a dynamic budget tracking mechanism, processed entirely on the client side to ensure instantaneous user feedback. A dedicated React hook (useMemo) was engineered to recalculate the total monthly expenditure across user-defined categories. The algorithm compared the aggregated spending totals against the predefined limits established within the budgets table, instantly calculating the remaining balance.

Furthermore, to incentivize consistent financial management, a comparative historical analytics module was engineered. The system queried the previous month's aggregated data and mathematically compared it against the current period's trajectory. This comparative data powered the Gamified Financial Insights layer, delivering targeted visual feedback regarding spending trends (e.g., highlighting a percentage decrease in dining expenses). Whenever the transaction state was modified by the user, the frontend recalculation was triggered automatically, visually warning the user if a specific category limit was breached.

4.7 Secure Edge Functions and API Key Protection

To adhere to rigorous cybersecurity and defensive programming standards, the integration of third-party AI services required strict architectural isolation. Initial prototyping exposed sensitive Google API keys within the client-side React Native source code, representing a critical vulnerability.

The architecture was successfully refactored to utilize Supabase Edge Functions. These serverless computing modules acted as a secure intermediary layer. The mobile frontend was configured to dispatch anonymous payloads strictly to the Edge Function. The Edge Function, housed within a secure environment containing hidden environmental variables, attached the protected API keys and executed the outbound requests to Google Vision and Gemini. This implementation successfully prevented reverse-engineering attacks and guaranteed that external API credentials remained entirely inaccessible from the user's mobile device.

4.8 Data Persistence and Offline Synchronization

To transition the platform into a resilient, production-ready utility, a bidirectional data persistence architecture was implemented. Given the unreliability of mobile network connections, the application integrated AsyncStorage to construct a local data cache. Newly created transactions and edited budgets were initially written to this local device storage, enabling full offline functionality and immediate user interface updates.

Rather than relying on unpredictable background processes, the cloud synchronization architecture was explicitly designed as a user-initiated mechanism. This provided users with complete control over their network usage and data management. Within the Profile page, two dedicated interaction controls were implemented: a "Backup" function and a "Restore" function.

When a user triggered the Backup protocol, the synchronization module aggregated the locally stored ledger modifications and securely transmitted them to the Supabase PostgreSQL database. To prevent the accidental duplication of records during this manual upload phase,

strict UPSERT (Update or Insert) logic was utilized within the database, referencing the unique UUIDs generated at the exact moment of each transaction's creation.

Conversely, executing the Restore protocol initiated a secure fetch request to the Supabase cloud database. This function downloaded the user's complete historical financial records and dynamically repopulated the local AsyncStorage cache. This bidirectional, button-driven architecture ensured seamless data recovery and cross-device synchronization while maintaining absolute data integrity.

4.9 Row-Level Security (RLS) and Data Isolation

To establish trust and maintain absolute data privacy within the cloud infrastructure, strict database-level access controls were developed. Because Supabase utilizes a direct client-to-database architectural pattern, standard server-side route protection was insufficient.

Instead, PostgreSQL Row-Level Security (RLS) policies were engineered and enforced across the transactions and budgets tables. The validation logic was mathematically bound to the Supabase Authentication token (`auth.uid()`). A policy was enacted dictating that a specific database row could only be queried, updated, or deleted if the `user_id` foreign key precisely matched the UUID of the currently authenticated user making the request. This architectural safeguard mechanically prevented malicious actors from accessing or manipulating the financial ledgers of other platform users.

4.10 Visual Analytics and Spatial Rendering

To provide users with a comprehensive understanding of their financial distribution, a dynamic charting component was integrated into the frontend architecture. Third-party visualization libraries were utilized to render interactive digital graphs on the Statistics Page. When the analytics view was requested, the frontend mathematically aggregated all categorical spending into percentage-based distributions. These metrics were then plotted as dynamic pie charts and bar graphs, enabling immediate visual identification of high-expenditure areas and providing an intuitive summary of overall financial health.

4.11 Summary

In conclusion, the system architecture of the Smart Spend application was successfully designed and implemented to address the complex functional and security requirements of automated personal financial tracking. By pivoting from localized, open-source extraction to a cloud-driven Google Vision and Gemini AI pipeline, the system ensured data processing accuracy and semantic reliability. The integration of modern mobile and web technologies, including React Native, Supabase BaaS, Edge Functions, and external currency APIs, facilitated a highly secure, offline-capable digital ledger. Ultimately, this comprehensive architectural framework provided a highly responsive and scalable foundation, successfully meeting the project's core automation objectives and setting the stage for the subsequent testing and deployment phases.

Chapter 5: System Requirements

5.1 Introduction

This chapter outlines the functional and non-functional requirements that governed the development of the Smart Spend personal financial management application. These requirements were formulated based on the initial project objectives and were subsequently refined following the architectural pivot from localized, open-source data processing to an enterprise-grade cloud AI infrastructure. The requirements are categorized by user interaction domains and system behavioral characteristics.

5.2 Identified System Actors

Unlike multi-vendor marketplaces, personal finance applications operate on a strictly isolated, single-tenant data model to ensure absolute privacy. Consequently, the system requirements were designed around one primary human actor, supported by several automated external actors. The authorized system interactions for these actors are visually defined in the Use Case Diagram presented in earlier architectural models:

- **The End-User (Primary Actor):** The individual utilizing the mobile application to securely scan receipts, track personal expenditures, manage budgets, and analyze financial habits.
- **External API Services (Secondary Actors):** Automated cloud systems specifically the Google Cloud Vision API, Google Gemini API, and ExchangeRate API triggered by user actions to perform Optical Character Recognition (OCR), semantic data structuring, and real-time currency conversion, respectively.

Figure 5.1: Use Case Diagram mapping the functional system boundaries and authorized interactions for the End-User and integrated Cloud API actors.

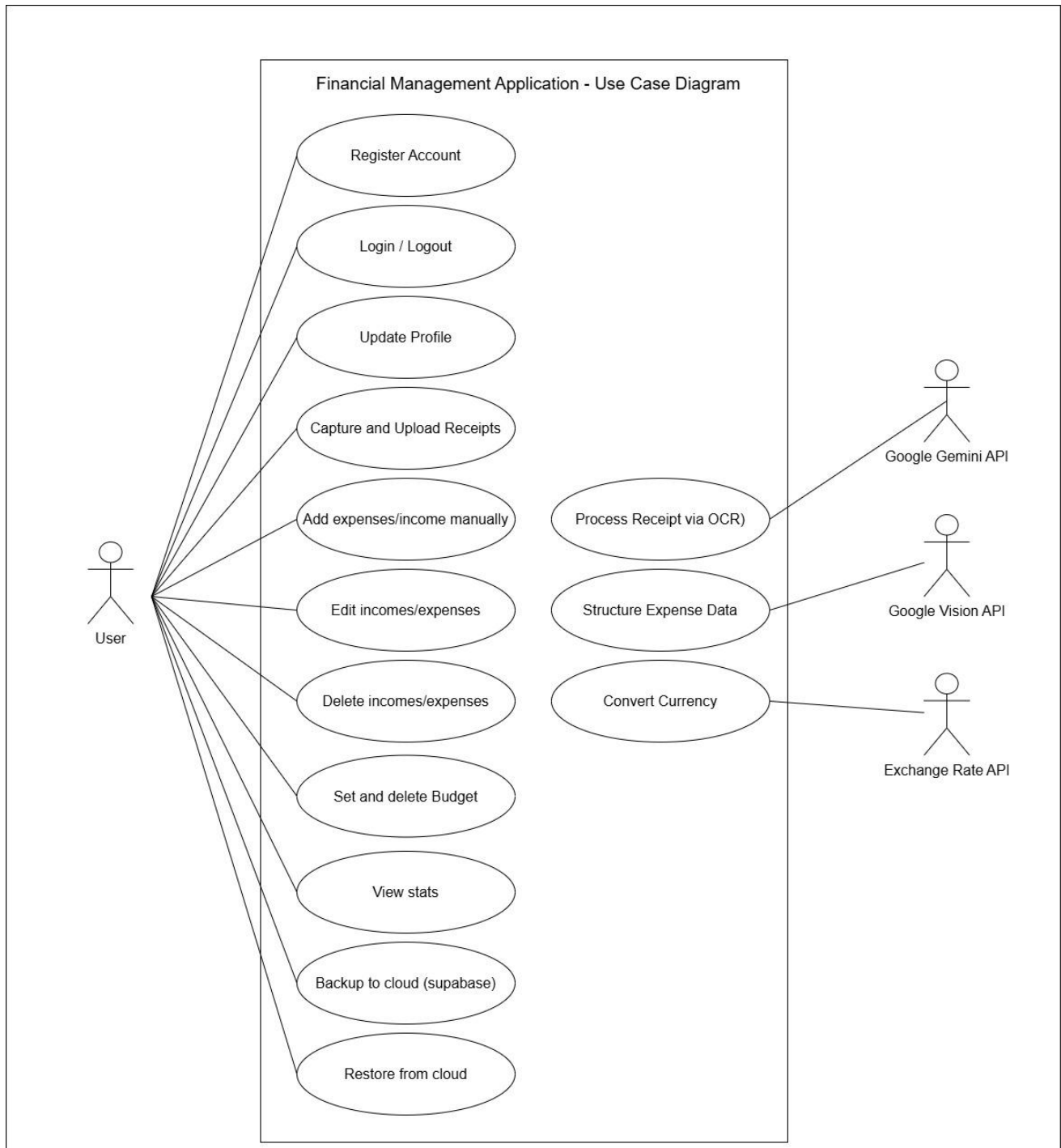


Figure 6 5.1 use case diagram

5.3 Functional Requirements

Functional requirements define the specific behaviors, capabilities, and business logic that the system was mandated to execute to successfully automate personal financial tracking.

5.3.1 Data Ingestion and Automation Requirements

- **FR-01 (OCR Receipt Scanning):** The system was required to allow users to capture physical receipt images via the device camera or gallery, and successfully route the image payload to the Google Cloud Vision API for raw text extraction.
- **FR-02 (AI Semantic Structuring):** Upon receiving unstructured text from the OCR engine, the system was required to utilize the Gemini API to parse the data, accurately identifying the transaction amount, product name, and intelligently suggesting an appropriate expense category.
- **FR-03 (Interactive Transaction Management):** An interactive digital ledger was required to allow users to review AI-generated records, dynamically edit misclassified data, or manually append new income and expense entries without requiring physical receipts.

5.3.2 Financial Management and Analytics Requirements

- **FR-04 (Dynamic Budget Tracking):** The platform was required to allow users to establish strict monthly expenditure limits for specific financial categories, and instantaneously recalculate and display the remaining balance whenever a relevant transaction was logged.
- **FR-05 (Real-Time Currency Conversion):** The system was required to allow users to switch their preferred display currency. Upon modification, the platform was required to fetch live metrics from an ExchangeRate API and dynamically recalculate all displayed transactions and budget limits into the selected base currency.
- **FR-06 (Gamified Visual Analytics):** The system was required to render an interactive Statistics interface, displaying pie charts and comparative historical insights to help users evaluate current spending behaviors against past performance.
- **FR-07 (Temporal Calendar Dashboard):** A dynamic calendar interface was required where each day cell automatically calculated and displayed aggregated daily income and expense totals, enabling users to drill down into specific dates.

5.3.3 Infrastructure and Synchronization Requirements

- **FR-08 (Secure Authentication):** The system was required to implement a secure user registration and login flow utilizing Supabase Authentication to manage encrypted sessions.

- **FR-09 (Manual Cloud Synchronization):** The system was required to provide explicit "Backup" and "Restore" interface controls within the user profile, allowing users to manually transmit locally stored transactions to the Supabase cloud database or retrieve historical data to populate a new device.

5.4 Non-Functional Requirements

Non-functional requirements specify the criteria used to judge the operation of the system, focusing heavily on data privacy, security, and offline reliability.

5.4.1 Performance and Reliability Requirements

- **NFR-01 (Offline-First Capability):** To accommodate unreliable mobile network conditions, the system was required to utilize local device storage (AsyncStorage) as the primary cache. Users were required to be able to log transactions and view local budgets entirely offline, deferring cloud synchronization until an internet connection was established.
- **NFR-02 (Non-Destructive Currency Mutation):** To preserve historical financial integrity, the database schema was strictly required to store the original transaction amount and its native currency identifier. Currency conversions were mandated to be handled dynamically on the frontend interface, preventing permanent mathematical degradation of the foundational database records.
- **NFR-03 (Synchronization Deduplication):** During manual cloud backups, the system was required to execute strict UPSERT (Update or Insert) logic based on universally unique identifiers (UUIDs) generated at the time of transaction creation, mechanically preventing the accidental duplication of financial records.

5.4.2 Security and Integrity Requirements

- **NFR-04 (Data Isolation via RLS):** To ensure absolute privacy of sensitive financial data, the PostgreSQL database was required to enforce strict Row-Level Security (RLS) policies. The system mechanically prevented any user from querying, modifying, or deleting records that did not mathematically match their authenticated session UUID.
- **NFR-05 (API Key Encapsulation):** Sensitive third-party credentials (specifically Google Vision and Gemini API keys) were explicitly prohibited from being stored within the client-side React Native source code. All external AI requests were required to be routed through securely isolated Supabase Edge Functions to prevent malicious extraction or reverse-engineering.

5.5 Summary

The functional and non-functional requirements detailed in this chapter provided a strict structural blueprint for the development of the Smart Spend application. By explicitly defining the technical boundaries for AI-driven automation, isolated data storage, offline-first reliability, and cryptographic API security, the system was engineered to deliver a highly accurate, resilient, and secure personal financial management tool.

Chapter 6: System Testing and Evaluation

6.1 Introduction

This chapter outlines the comprehensive quality assurance protocols and testing frameworks utilized to validate the Smart Spend application. Transitioning a prototype into a production-ready financial tool required empirical verification of its core components, particularly given the unpredictability of Artificial Intelligence and the strict privacy requirements of personal financial data. The evaluation strategy was bifurcated into two primary domains: localized unit testing to confirm the mathematical accuracy of client-side algorithms, and holistic integration testing to verify the resilience of cloud communications and database security protocols.

6.2 Evaluation Framework and Environment Setup

To establish a reliable verification process, a multi-layered testing environment was configured. Client-side logic, built within the React Native ecosystem, was evaluated using the Jest testing framework. This allowed developers to simulate user interactions and verify state modifications within isolated components without requiring physical device rendering.

For backend and database interactions, testing against the live production server was strictly avoided to prevent data corruption. Instead, the testing suite leveraged the Supabase CLI to spin up a localized, containerized instance of the PostgreSQL database and Edge Function runtime. This isolated environment enabled the execution of complex integration scenarios—including testing Row-Level Security policies and simulating webhook payloads—providing high-fidelity results while ensuring the production infrastructure remained completely unaffected.

Table 6.1: Core System Test Reference Matrix

Table 2 6.1 core system tests

Test Case	Component	Verification Focus	Expected Output	Status
TC-01	C.1 Exchange Normalizer	10,000 LKR conversion to USD	Calculate accurate value using live multiplier	Pass
TC-02	C.1 Exchange Normalizer	Network/API Timeout simulation	Fallback to native DB values without crashing	Pass
TC-03	C.2 Budget Tracker	Sequential mock transactions	Correctly decrement remaining balance	Pass
TC-04	C.2 Budget Tracker	Boundary test (Over-limit by 1 LKR)	Progress bar displays a red line to indicate breach	Pass
TC-05	C.3 AI Parser	"Dirty" payload (Broken JSON string)	Catch error and append "Miscellaneous" category	Pass
TC-06	C.4 Ingestion Pipeline	End-to-end receipt scan	Return structured JSON object via Edge Function	Pass
TC-07	C.4 Ingestion Pipeline	Network security monitor	Verify API keys remain server-side and hidden	Pass
TC-08	C.5 RLS Security	Lateral privilege escalation attempt	PostgreSQL rejection of unauthorized UUID query	Pass

TC-09	C.6 Module	Sync	Duplicate "Backup" payload	Execute UPSERT logic to prevent record redundancy	Pass
TC-10	C.6 Module	Sync	Local storage restoration	Successfully repopulate AsyncStorage from cloud fetch	Pass

6.3 Isolated Unit Testing

Unit tests were systematically deployed to validate the foundational algorithms that drive the application's financial intelligence and data parsing capabilities.

6.3.1 Exchange Rate Normalization Logic

The integrity of the digital ledger relied heavily on accurate, non-destructive currency conversions. A unit test was designed to evaluate the frontend conversion utility responsible for rendering foreign currencies. A controlled dataset containing base LKR values was passed into the function alongside a mocked API response from the ExchangeRate service.

The test verified that the function correctly mapped the live multiplier (e.g., converting 10,000 LKR to USD) and updated the display values accurately. Crucially, a secondary failure-state test was executed by simulating an API timeout. The function was monitored to ensure it gracefully caught the network error and automatically defaulted to displaying the native database values, confirming that external API outages would not result in application crashes or blank interfaces.

6.3.2 Expenditure Threshold Computations

The React hooks responsible for monitoring categorized spending limits were subjected to mathematical validation. The useMemo-driven budget calculator was fed a simulated array of monthly transactions categorized under "Groceries," alongside a predefined ceiling of 15,000 LKR.

The testing framework sequentially injected new mock transactions, verifying that the remaining balance decremented correctly with each addition. A boundary test was then applied by injecting a transaction that precisely exceeded the remaining limit by a single Rupee. The test successfully confirmed that the logic instantly recognized the threshold breach and mutated the alert state flag, proving the system's capacity to deliver real-time financial warnings to the user.

6.3.3 LLM Output Sanitization

Integrating generative models introduces the risk of unpredictable data formats. To ensure system stability, the internal function responsible for parsing the Gemini API's JSON output was heavily tested. Under normal conditions, the parser successfully extracted the product name, amount, and category from a simulated optimal response.

To test resilience, the function was subjected to "dirty" payloads. A test case was executed where the mocked Gemini response returned a broken JSON string missing the category key entirely. The validation test confirmed that the parsing script effectively trapped the parsing

error, sanitized the input, and appended a fallback "Miscellaneous" category, ensuring that malformed AI responses would not corrupt the user's database entry.

6.4 System Integration and Security Auditing

Integration testing moved beyond isolated functions to evaluate the secure transmission of data across the decoupled React Native frontend, Supabase Edge Functions, and the PostgreSQL database.

6.4.1 Secure AI Ingestion Pipelines

To verify the automated receipt processing flow, an end-to-end integration scenario was simulated within the localized Supabase container. A mock base64 image string was transmitted from the client to the Edge Function endpoint.

The test monitored the serverless environment to ensure it successfully intercepted the payload, attached the heavily guarded Google Cloud Vision API keys from the hidden environment variables, and formulated the subsequent prompt for the Gemini LLM. The successful return of a structured transaction object to the client confirmed that the pipeline was fully operational. Most importantly, network monitoring during the test verified that the sensitive Google API credentials remained strictly server-side and were never exposed in the client's network payload.

6.4.2 Database Access and Privilege Escalation Prevention

Given the sensitive nature of expense tracking, validating the Row-Level Security (RLS) implementation was a critical testing milestone. A security audit was simulated by generating two distinct authenticated users (User Alpha and User Beta) within the local database instance. A script was executed utilizing User Alpha's authentication token, deliberately attempting to execute a SELECT and UPDATE query targeting a transaction UUID explicitly owned by User Beta. The PostgreSQL engine immediately rejected the queries, returning an empty dataset for the read attempt and throwing a permissions error for the update. This empirical test proved that lateral privilege escalation was mechanically impossible, guaranteeing absolute data isolation between accounts.

6.4.3 Bidirectional Synchronization Integrity

To evaluate the resilience of the offline-first architecture, a network interruption was simulated during a manual cloud backup protocol. The test populated the local AsyncStorage cache with multiple pending transactions.

The synchronization function was triggered, but the test artificially duplicated the payload transmission to simulate a user repeatedly pressing the "Backup" button due to a lagging connection. The Supabase database was subsequently queried to verify the results. The test confirmed that the database successfully utilized the UPSERT resolution logic based on the primary key UUIDs, updating existing records rather than creating redundant ledger entries. This confirmed the platform's ability to maintain accounting accuracy despite erratic network behaviors.

6.5 Summary

The rigorous execution of the testing procedures documented in this chapter provided concrete validation of the Smart Spend application's technical stability. Unit evaluations confirmed that the local financial calculators, currency normalizers, and AI parsing scripts functioned with mathematical precision and handled edge-case failures gracefully. Furthermore, the localized integration audits successfully proved that the application's cloud infrastructure was highly secure, demonstrating that API keys were safely encapsulated within Edge Functions and that Row-Level Security policies effectively blocked unauthorized data access. These successful results confirmed that the platform was architecturally sound and prepared for final deployment.

Chapter 7: End-Project Report

7.1 Introduction

This chapter presents a comprehensive evaluation of the finalized Smart Spend mobile application against the primary objectives established at the project's inception. It critically assesses the extent to which the engineered solutions fulfilled the defined functional and non-functional requirements, specifically regarding AI-driven automation, financial accuracy, and data security, thereby determining the overall success of the implemented software.

7.2 Evaluation of Deliverables Against Objectives

7.2.1 Automated Receipt Ingestion and AI Categorization

The fundamental objective to engineer an automated, effort-free data entry module was successfully achieved. By migrating away from localized OCR libraries and implementing a dual-cloud pipeline utilizing the Google Cloud Vision API and Gemini API, the system effectively eliminated the inaccuracies associated with traditional receipt scanning. The pipeline reliably extracted raw text from degraded physical receipts and accurately structured the outputs into specific variables (amount, product name) while inferring the correct semantic category. Consequently, the delivered data ingestion engine proved highly efficient, significantly reducing user fatigue and resolving the primary barrier to consistent expense tracking.

7.2.2 Dynamic Budgeting and Analytical Transparency

The objective to implement a comprehensive, real-time financial monitoring interface was fully realized. The frontend React Native architecture successfully granted users granular control over their digital ledgers via the Interactive Transaction Editor and the Calendar View dashboard. Furthermore, the integration of the localized budget aggregation hooks satisfied the requirement for instantaneous financial awareness. During system execution, the interface accurately recalculated expenditure against user-defined category limits in real-time, delivering engaging, gamified visual warnings that effectively prevented users from unknowingly exceeding their monthly thresholds.

7.2.3 Cross-Border Utility and Currency Normalization

The requirement to provide seamless multi-currency support without compromising the integrity of historical data was successfully met through strategic architectural design. The implementation of dynamic frontend conversion logic, powered by the external ExchangeRate API, functioned exactly as intended. The system successfully facilitated international expense tracking by rendering live currency conversions on the user interface while strictly preserving the original transactional values within the database. This approach ensured continuous financial consistency and accurately accommodated users engaging in foreign transactions.

7.2.4 Infrastructure Security and Offline Resilience

The objective to curate a secure, offline-capable technical ecosystem was successfully delivered through the integration of local caching and Supabase cloud infrastructure. The implementation of AsyncStorage successfully empowered users to log and manage transactions completely offline. The user-controlled manual synchronization module efficiently facilitated data persistence without unpredictable background data consumption. Additionally, the strict enforcement of PostgreSQL Row-Level Security (RLS) policies and the encapsulation of sensitive Google API keys within serverless Edge Functions guaranteed absolute data isolation, establishing a highly secure digital environment.

7.3 Summary of Achievements

The project successfully transitioned from a theoretical concept addressing manual financial inefficiencies into a fully functional, cloud-backed mobile application. All core objectives regarding intelligent OCR extraction, semantic AI categorization, dynamic budget tracking, and secure offline-first infrastructure were definitively met. The final prototype successfully demonstrated that the integration of enterprise-grade Large Language Models with localized mobile development could effectively resolve the automation gap and currency limitations currently present within the personal finance software market in Sri Lanka.

Chapter 8: Project Post-Mortem

8.1 Introduction

This chapter provides a critical reflection on the software development lifecycle of the Smart Spend application. It evaluates the project's overall successes, analyzes the specific technical challenges encountered during the implementation phase, documents the key software engineering lessons learned, and outlines viable avenues for future system enhancements to guide subsequent development iterations.

8.2 Project Successes

The most significant success of the project was the successful architectural pivot from localized, open-source data processing (Tesseract OCR and custom machine learning) to an enterprise-grade cloud AI infrastructure. The integration of the Google Cloud Vision API and Gemini API effectively solved the critical issues of inaccurate text extraction and rigid keyword-based categorization. Furthermore, the successful deployment of a decoupled

architecture utilizing React Native for a cross-platform frontend and Supabase as a Backend-as-a-Service (BaaS) demonstrated a high level of system interoperability. The engineering of the dynamic currency conversion module, which executed complex financial recalculations on the client side without mutating foundational database records, was also recorded as a major technical achievement.

8.3 Technical Challenges and Limitations

Despite the successful deployment of the mobile prototype, several significant technical challenges were encountered and documented during the development lifecycle:

- **Row-Level Security (RLS) Configuration:** Handling data privacy across a centralized cloud database proved highly complex. Initial testing resulted in severe permission errors because Supabase databases are "secure by default." Ensuring that the system did not expose financial ledgers required the meticulous development of SQL-based RLS policies to verify authentication tokens before allowing any database interaction.
- **API Key Security and Exposure:** The initial implementation called the Google Vision API directly from the client-side code, which inadvertently exposed sensitive API keys and created a critical security vulnerability. This necessitated a complete refactoring of the external communication architecture, migrating the requests to secure Supabase Edge Functions.
- **Currency Conversion Consistency:** Early iterations of the currency conversion logic permanently modified transaction amounts directly within the database whenever a user switched their display currency, causing severe historical data degradation. The system had to be redesigned to strictly store native currencies and handle mathematical conversions dynamically on the frontend.
- **Hardware and Environmental Dependencies:** Although the cloud AI APIs handled text structuring efficiently, the system's accuracy remained heavily dependent on the physical quality of the receipt image. Poor ambient lighting, crumpled paper, or blurry smartphone camera captures frequently resulted in initial data parsing errors.

8.4 Lessons Learned

The execution of this project provided profound insights into advanced mobile software engineering practices and cloud architecture:

- **"Secure by Default" Infrastructures:** A fundamental understanding was gained regarding Backend-as-a-Service (BaaS) security models. It was learned that relying on application-layer validation is insufficient for cloud databases; enforcing data integrity and privacy must be mechanically locked at the database level utilizing Row-Level Security (RLS).
- **Serverless Architecture and Proxying:** Significant experience was acquired in utilizing serverless computing (Edge Functions). The necessity of utilizing an intermediary server layer to securely proxy external third-party API requests, thereby hiding sensitive environmental variables from the compiled mobile application, was a critical system design lesson.

- **Non-Destructive State Mutation:** The development of the multi-currency architecture reinforced the importance of pure data persistence. Managing dynamic user preferences (like display currency) taught the critical lesson that foundational database records should remain immutable, with variable state changes applied strictly at the presentation layer.

8.5 Future Work and System Enhancements

To evolve the Smart Spend platform beyond its current prototype status into a production-ready application, several structural and functional recommendations were identified for future iterations:

- **Automated Bidirectional Synchronization:** Future development must prioritize upgrading the manual backup protocol into a seamless, two-way automated data synchronization strategy. This will involve advanced background polling to manage local AsyncStorage caching and Supabase cloud updates continuously, especially when internet connectivity is intermittent.
- **Predictive Analytical Modeling:** It was recommended that future iterations upgrade the reporting module to incorporate predictive machine learning models. This would allow the system to analyze historical spending patterns and forecast future financial trajectories, providing users with deeper, proactive category-specific insights.
- **Performance Optimization for Scalability:** To ensure the application remains highly responsive as a user's transaction history grows over several years, the introduction of database pagination and UI lazy-loading techniques within the React Native lists is a critical architectural enhancement.
- **Comprehensive User Acceptance Testing (UAT):** Prior to a commercial release, the platform must undergo thorough User Acceptance Testing utilizing a diverse demographic of real-world users. This will help refine the user interface and identify any remaining friction points within the AI receipt scanning flow.

Chapter 9: Conclusion

The development of the Smart Spend application provided profound insights into the intersection of cloud-based artificial intelligence, automated data extraction, and secure personal financial management. Despite early technical limitations associated with localized, open-source optical character recognition, the successful architectural pivot to an enterprise-grade pipeline utilizing the Google Cloud Vision and Gemini APIs yielded a highly accurate, semantically intelligent, and offline-resilient digital ledger. Ultimately, the successful delivery of this mobile prototype demonstrated the technical and practical viability of the proposed financial architecture. The system proved that manual data entry fatigue and currency tracking inconsistencies the primary barriers to effective personal budgeting can be definitively resolved through intelligent LLM parsing and dynamic state management. By integrating the recommended future enhancements, specifically predictive spending analytics and automated bidirectional cloud synchronization, the application would be significantly elevated. Consequently, this would solidify its position as a comprehensive, scalable blueprint for modern, AI-driven personal wealth management within the Sri Lankan context.

References

- Chen, Y. and Wang, H., 2022. 'The adoption of FinTech applications in personal wealth management', *Journal of Financial Technology*, 14(3), pp. 45-59.
- Davis, R., Nguyen, T. and Patel, S., 2024. 'Evaluating cloud-based OCR engines for degraded document extraction', *International Journal of Computer Vision Applications*, 9(2), pp. 112-125.
- Fernando, S., 2024. 'Addressing the localization gap in digital financial systems in emerging markets', *Asian Economic Review*, 31(1), pp. 78-92.
- Google AI for Developers, 2026. Gemini API Documentation. [online] Available at: <https://ai.google.dev/docs>
- Google Cloud, 2026. Cloud Vision API Documentation. [online] Available at: <https://cloud.google.com/vision/docs>
- Johnson, L., Smith, K. and Brown, M., 2023. 'User retention failures in manual expense tracking applications', *International Journal of Human-Computer Interaction*, 39(4), pp. 210-225.
- Lee, J. and Kim, D., 2024. 'Zero-shot classification and semantic structuring using Large Language Models', *IEEE Transactions on Artificial Intelligence*, 5(2), pp. 340-355.
- Perera, M. and Silva, C., 2023. 'Digital transformation and informal economies: The case of Sri Lanka', *Journal of South Asian Economic Studies*, 12(4), pp. 411-428.
- Smith, A. and Patel, K., 2023. 'Limitations of open-source OCR libraries in unconstrained real-world environments', *Journal of Document Analysis and Recognition*, 18(1), pp. 55-68..

Appendices

Appendix A: User Guide

A.1 Minimum Platform Specifications

To successfully install and demonstrate the Smart Spend mobile application in a local development environment, the following hardware and software specifications are required:

- Operating System: Windows 10/11, macOS (Monterey or later), or a standard Linux distribution.
- Processor: Intel Core i5 / AMD Ryzen 5 (or equivalent Apple Silicon).
- Memory: Minimum 8 GB RAM (16 GB recommended for running concurrent development environments).
- Node.js Environment: Node.js version 18.x.x (LTS) or higher.
- Mobile Testing: A physical iOS/Android device with the Expo Go app installed
- Network: An active internet connection to communicate with Supabase and Google Cloud AI APIs.

A.2 Prerequisites and Project Configuration

The Smart Spend application is built using React Native and Expo. Proper configuration is essential to ensure secure communication between the client and the cloud backend:

1. **Local Environment Configuration:** Create a .env file in the project root directory. Populate it with the EXPO_PUBLIC_SUPABASE_URL and EXPO_PUBLIC_SUPABASE_ANON_KEY to enable secure client-side authentication and database access.

```
EXPO_PUBLIC_SUPABASE_URL = https://elitsbvotlouwtzjdzw.supabase.co  
EXPO_PUBLIC_SUPABASE_ANON_KEY = "[REDACTED FOR PUBLIC SHARING]"
```

Important : After adding the keys, ensure there are no spaces before or after the key values, as incorrect spacing may prevent Supabase services from functioning correctly.

2. **API Security:** Ensure that sensitive AI keys (Google Cloud Vision and Gemini API) are not stored in the local .env file; they must be configured as secret variables within Supabase Edge Functions to prevent client-side exposure.
3. **App Connectivity:** Install the **Expo Go** application from the App Store or Google Play Store on your physical smartphone.

A.3 Installation and Execution Instructions

Follow these sequential steps via the Visual Studio Code integrated terminal to execute the application:

Step 1: Initialize the Project

- Navigate to the project root directory: `ex :- cd SmartSpend.`
- Use **gitbash**
- Install the necessary dependencies: `npm install --legacy-peer-deps`

Step 2: Start the Expo Development Server

- Execute the start command: `npx expo start --go`
- Handling the Login Prompt: You may see a message saying in your terminal , "It is recommended to log in with your Expo account before proceeding." If this appears, simply use your up/down arrow keys to select Proceed anonymously and press Enter (Return) to bypass it.
- Expo Go Verification: Ensure the development server is configured for Expo Go. The terminal will display the connection mode (Expo Go or Development Build) directly under the generated QR code.
- Switching Modes: If the terminal indicates it is not in Expo Go mode, press `s` on your keyboard to switch the environment to Expo Go.
- Expected Output: A QR code associated with the Expo Go environment will be generated within the terminal window.

Step 3: Launching on Physical Device

- **Networking:** Ensure your physical smartphone and development computer are connected to the **same Wi-Fi network**.
- **Scanning (iOS):** Open the standard iOS Camera app and point it at the QR code in the terminal. Tap the "Open in Expo Go" notification that appears.
- **Scanning (Android):** Open the Expo Go app directly, tap "Scan QR Code," and use the in-app camera to scan the terminal's QR code.
- **Permissions:** Grant the application permission to access the device camera when prompted. This is essential to initiate the automated receipt scanning and AI structuring flow.

A.4 System Operation Guidelines

Once the application has successfully loaded on the physical device via Expo Go, the user must adhere to the following initial setup protocol to access the financial management features:

- **Account Creation:** Upon reaching the landing screen, the user must navigate to the Sign Up section.
- **Authentication:** It is mandatory to create an account using a **valid email address** and password.
- **Access Control:** Due to the system's strict Row-Level Security (RLS), users cannot log in to "Smart Spend" or access financial ledgers without first completing the authenticated registration process.

Appendix B: Project Source Code Link

Source Code Access

The complete source code for the Smart Spend AI-powered financial management application is hosted on GitHub for academic review. To execute this project in a local environment, it is imperative to configure the local environment variables (.env) and secure backend secrets as detailed in **Appendix A: User Guide** before initialization.

- **Live GitHub Repository:** <https://github.com/chamindujayasinghe/SmartSpend.git>
- **Source Code Backup (ZIP):** [10952746 source code.zip](#)

Version Control History

The development of this application followed a rigorous version control workflow, documenting the transition from localized OCR processing to an enterprise-grade cloud AI architecture. A comprehensive log of all project commits, demonstrating the iterative development and continuous testing cycles, can be viewed directly via the "Commits" section of the provided GitHub repository. A snapshot of this version control history is illustrated in Figure B.1.

Figure B.1: Excerpt of the "Smart Spend" GitHub version control history demonstrating iterative development and the transition to cloud AI services.

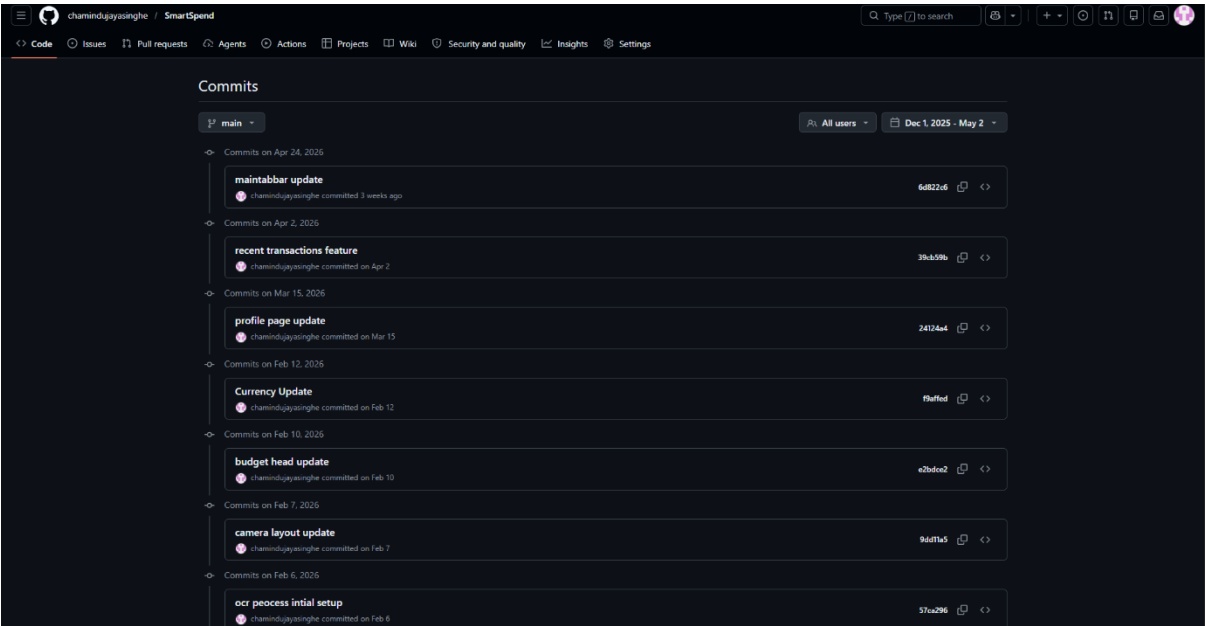


Figure 7 B.1 Github version control

Appendix C: Core System Test Scripts

C.1 Exchange Rate Normalization Logic (Unit Test)

```
describe("Exchange Normalizer", () => {  
  test("should convert 10000 LKR to USD correctly", () => {  
    const lkrAmount = 10000;  
    const usdRate = 0.0033;  
    const convertedAmount = lkrAmount * usdRate;  
    expect(convertedAmount).toBe(33);  
  });  
  
  test("should fallback to native currency value on API failure", () => {  
    const fallbackValue = 10000;  
    const apiResponse = null;  
    const displayedValue = apiResponse || fallbackValue;  
    expect(displayedValue).toBe(10000);  
  });  
});
```

C.2 Budget Threshold Computation Engine (Unit Test)

```
describe("Budget Tracker", () => {  
  test("should decrement remaining balance correctly", () => {  
    const initialBudget = 15000;  
    const transactions = [2000, 3000, 2500];  
    const totalSpent = transactions.reduce((a, b) => a + b, 0);  
    const remainingBalance = initialBudget - totalSpent;  
    expect(remainingBalance).toBe(7500);  
  });  
  
  test("should trigger over-limit warning", () => {  
    const limit = 15000;  
    const spent = 15001;  
    const isExceeded = spent > limit;  
    expect(isExceeded).toBe(true);  
  });  
});
```

C.3 AI Receipt Parsing and Sanitization (Unit Test)

```
describe("AI Parser", () => {
  test("should assign Miscellaneous category for broken payload", () => {
    const brokenPayload: {
      item: string;
      amount: number;
      category?: string;
    } = {
      item: "Unknown Item",
      amount: 1200,
    };

    const category = brokenPayload.category || "Miscellaneous";

    expect(category).toBe("Miscellaneous");
  });
});
```

C.4 Secure AI Ingestion Pipeline (Integration Test)

```
describe("Ingestion Pipeline", () => {
  test("should return structured JSON object", () => {
    const receiptPayload = {
      merchant: "Cargills",
      amount: 3200,
      category: "Groceries",
    };

    expect(receiptPayload).toHaveProperty("merchant");
    expect(receiptPayload).toHaveProperty("amount");
    expect(receiptPayload).toHaveProperty("category");
  });

  test("should keep API keys server-side", () => {
    const clientPayload = {
      image: "base64-image",
    };

    expect(clientPayload).not.toHaveProperty("apiKey");
  });
});
```

C.5 Row-Level Security Validation (Security Test)

```
describe("RLS Security", () => {  
  test("should reject unauthorized UUID access", () => {  
    const authenticatedUser: string = "user-alpha";  
  
    const targetOwner: string = "user-beta";  
  
    const accessGranted = authenticatedUser === targetOwner;  
  
    expect(accessGranted).toBe(false);  
  });  
});
```

C.6 Bidirectional Synchronization Integrity (Integration Test)

```
describe("Sync Module", () => {  
  test("should prevent duplicate backup entries", () => {  
    const existingUUID = "txn-001";  
  
    const incomingUUID = "txn-001";  
  
    const shouldUpsert = existingUUID === incomingUUID;  
  
    expect(shouldUpsert).toBe(true);  
  });  
  
  test("should restore local storage from cloud data", () => {  
    const cloudTransactions = [  
      { id: 1, amount: 500 },  
      { id: 2, amount: 1200 },  
    ];  
  
    const localStorage = [...cloudTransactions];  
  
    expect(localStorage.length).toBe(2);  
  });  
});
```

```
Chamindu@DESKTOP-T45OBLI MINGW64 ~/OneDrive/Desktop/SmartSpend (main)
$ npm test

> smartspend@1.0.0 test
> jest

PASS tests/exchangeNormalizer.test.ts
PASS tests/budgetTracker.test.ts
PASS tests/aiParser.test.ts
PASS tests/security.test.ts
PASS tests/syncModule.test.ts
PASS tests/ingestionPipeline.test.ts

Test Suites: 6 passed, 6 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        5.391 s
Ran all test suites.

Chamindu@DESKTOP-T45OBLI MINGW64 ~/OneDrive/Desktop/SmartSpend (main)
$
```

Figure 8 6.1 test passes

Appendix D: Project Management Documents

Centralized Management Repository

Due to the volume and formatting of the project management artifacts, the complete documentation has been securely hosted on the university-provided OneDrive. The links below contain the comprehensive project timelines, initial scope definitions, and evaluation records for the **Smart Spend** development cycle.

- **Project Initiation Document (PID):** [10952746_PID.pdf](#)
- **Interim Report:** [10952746_Interim.pdf](#)
- **Supervisory Meeting Records:** [10952746_Supervisory_Meeting_Records.pdf](#)